

Cloud Computing
Semester 2026-III
Delivery No. 3 — Cloud Architecture & Scalability

Eng. Carlos Andrés Sierra, M.Sc.

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the third delivery of the *Cloud Computing* course project!

In **Delivery 1**, you defined what the system must do and under what operational conditions. In **Delivery 2**, you designed a complete, cloud-independent software architecture. Now, the course shifts from software architecture to **cloud architecture**. The central question for this delivery is: **How do we deploy and operate the software architecture under the specified workload?**

This is where your architectural decisions meet reality. You will translate every service, database, queue, and API from your software architecture into concrete cloud infrastructure, justify every instance type and auto-scaling policy with numbers from your workload model, and demonstrate that your architecture can survive a $20\times$ growth in traffic.

General Delivery Definition: Cloud architecture design involves translating a cloud-independent software architecture into a concrete infrastructure deployment plan on a specific cloud provider. In this delivery, teams produce a complete cloud architecture covering compute, networking, data, communication, operations, and security — supported by quantitative capacity planning, a growth scenario analysis, and a monthly cost estimate. Every infrastructure decision must be justified by reference to the workload model from Delivery 1 and the service definitions from Delivery 2.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavir-guezs@udistrital.edu.co*.

Delivery Scope and Objectives

- **Cloud Architecture Diagram:** Translate the software architecture into a complete cloud infrastructure diagram using AWS services.
- **Compute Design:** Define containers or VMs, instance types, counts, stateless/stateful considerations, and auto-scaling policies for each service.
- **Networking Design:** Design the VPC, subnet structure, load balancing, and security boundaries.
- **Data Infrastructure:** Select managed database services, object storage, caching layers, and backup strategies.
- **Communication Infrastructure:** Map message queues, event buses, and asynchronous workers to managed cloud services.
- **Operations:** Define logging, monitoring, metrics collection, and health check strategies.
- **Security:** Define authentication, authorization, secrets management, and network security controls.
- **Growth Scenario:** Analyze how the architecture must evolve to handle a 20× increase in workload.
- **Cost Analysis:** Estimate the monthly AWS cost for the proposed architecture at initial and growth-scale workloads.

Required Artifacts and Methodology

1. Cloud Architecture Diagram:

- Produce a detailed AWS architecture diagram mapping every service from Delivery 2 to specific AWS resources
- The diagram must show: VPC and subnet structure, compute resources (ECS/EKS/EC2), managed databases (RDS/DynamoDB/ElastiCache), load balancers (ALB/NLB), queues and event buses (SQS/SNS/EventBridge), storage (S3), API Gateway, and monitoring (CloudWatch)
- Use standard AWS architecture diagram conventions with official AWS service icons
- Label every component with: service name, instance type or tier, and estimated instance count at average load

2. Compute Design:

- For each service defined in Delivery 2, specify:

- *Compute technology*: ECS (Fargate or EC2 launch type), EKS, or EC2 — with justification
- *Instance type or task size*: vCPU and memory allocation per instance/task
- *Stateless or stateful*: Justify the classification; stateful services require additional design attention
- *Minimum and maximum instances*: Under normal and peak load, based on your workload model
- *Auto-scaling policy*: Metric used for scaling (CPU utilization, request count, queue depth), scale-out threshold, scale-in threshold, and cooldown period
- Show instance count calculations: given the peak requests/hour from Delivery 1, how many instances are needed? Show the math.

3. Networking Design:

- *VPC Design*: Define the VPC CIDR range, Availability Zones used, and subnet structure
- *Public Subnets*: Components accessible from the internet (API Gateway, load balancers, NAT gateways)
- *Private Subnets*: Internal services, databases, and workers not directly exposed to the internet
- *Load Balancing*: Application Load Balancer (ALB) or Network Load Balancer (NLB) placement, listener rules, and target group configuration
- *Security Groups and NACLs*: Define ingress and egress rules for each tier (web, application, data)
- *Multi-AZ*: Specify which components are deployed across multiple Availability Zones and justify the decision

4. Data Infrastructure:

- For each service's data store defined in Delivery 2, specify the AWS managed service and configuration:
 - *Relational databases*: Amazon RDS engine (PostgreSQL/MySQL), instance class, Multi-AZ deployment, read replicas
 - *NoSQL databases*: Amazon DynamoDB table design, partition key strategy, capacity mode (on-demand or provisioned), GSIs
 - *Caching*: Amazon ElastiCache (Redis or Memcached), node type, cluster size, eviction policy
 - *Object storage*: Amazon S3 bucket configuration, storage class, lifecycle policies
 - *Backup strategy*: Automated backups, retention period, point-in-time recovery, and cross-region replication

5. Communication Infrastructure:

- For each asynchronous communication pattern defined in Delivery 2, specify:
 - *Amazon SQS*: Queue type (standard or FIFO), visibility timeout, message retention, dead-letter queue configuration
 - *Amazon SNS*: Topic configuration, subscription protocols, fan-out patterns
 - *Amazon EventBridge*: Event bus configuration, rules, and targets
- Define worker services that consume from queues and their auto-scaling policies based on queue depth (ApproximateNumberOfMessagesVisible)

6. Operations:

- *Logging*: Amazon CloudWatch Logs log groups per service, log retention policy, structured logging format
- *Monitoring*: Amazon CloudWatch dashboards, key metrics per service (latency, error rate, throughput, saturation)
- *Alerting*: CloudWatch Alarms for critical thresholds with SNS notification targets
- *Health Checks*: ALB health check endpoints per service, ECS/EKS liveness and readiness probes
- *Distributed Tracing*: AWS X-Ray integration for request tracing across services

7. Security:

- *Authentication*: Amazon Cognito User Pools or custom JWT-based authentication; define the token flow
- *Authorization*: IAM roles and policies per service (least-privilege principle); define role assignments
- *Secrets Management*: AWS Secrets Manager or Parameter Store for database credentials, API keys, and certificates
- *Network Security*: Security group rules, VPC endpoints for AWS services, WAF rules on API Gateway or ALB
- *Data Encryption*: Encryption at rest (KMS) and in transit (TLS 1.2+) for all data stores and communication channels

8. Growth Scenario — 20× Workload Increase:

At this point in the semester, a new requirement is introduced:

The company has grown by a factor of 20.

Initial workload: [your average req/hour from Delivery 1]
Growth workload: [20× the initial value]

Students must explain how their architecture changes under this growth scenario. For each major component category, answer:

- *Compute*: How many additional instances are needed? Does the instance type need to change? Are there any bottlenecks that cannot be solved by horizontal scaling alone?
- *Databases*: Can the current database configuration handle $20\times$ the write and read load? Are read replicas sufficient, or is sharding or a different data store required?
- *Networking*: Are there any network throughput or connection limit bottlenecks at $20\times$ load?
- *Communication*: Do queues need higher throughput configurations? Are there any consumer scaling limits?
- *Cost*: How does the monthly cost change at $20\times$ workload?

9. Cost Analysis:

Estimate the monthly AWS cost for the proposed architecture under two scenarios: **initial workload** and **$20\times$ growth workload**. Organize the cost estimate by category:

Category	Initial (\$/month)	$20\times$ Growth (\$/month)
Compute (ECS/EKS/EC2)		
Databases (RDS/DynamoDB)		
Caching (ElastiCache)		
Storage (S3)		
Networking (data transfer, NAT)		
Messaging (SQS/SNS/EventBridge)		
Monitoring (CloudWatch)		
Security (WAF, Secrets Manager)		
Total		

Use the **AWS Pricing Calculator** (calculator.aws) to estimate costs. All estimates must include the instance types, quantities, and usage assumptions used in the calculation.

Deliverables and Documentation

1. Cloud Architecture & Capacity Plan (PDF):

- *Executive Summary*: Overview of the cloud architecture approach and key infrastructure decisions
- *Cloud Architecture Diagram*: Complete AWS architecture diagram with all components labeled
- *Compute Design*: Instance type and count justification per service with scaling calculations

- *Networking Design*: VPC diagram, subnet structure, and security boundary definitions
- *Data Infrastructure*: Managed service selection and configuration per service
- *Communication Infrastructure*: Queue and event bus configuration with consumer scaling
- *Operations Plan*: Logging, monitoring, alerting, and health check definitions
- *Security Design*: Authentication, authorization, secrets, and network security controls
- *Growth Scenario Analysis*: 20× workload impact per component with architectural changes
- *Cost Analysis*: Monthly cost estimates for initial and 20× growth scenarios

2. Repository Update:

- Add the Cloud Architecture & Capacity Plan PDF to the `delivery_3/` folder
- Include architecture diagrams as source files in `delivery_3/diagrams/`
- Include AWS Pricing Calculator export or screenshots in `delivery_3/cost/`
- Update the `README.md` to reflect the current project status

Submission Requirements

- The PDF report file must follow the naming convention: `groupN_delivery3.pdf`, where N is the team number (e.g., `group1_delivery3.pdf`, `group3_delivery3.pdf`)
- Submit the complete Cloud Architecture & Capacity Plan as a single PDF through the designated course platform
- Include the GitHub repository link with all diagrams, source files, and cost documentation
- All documentation must be in **English** and follow professional technical writing standards
- All instance counts and sizing decisions must include the arithmetic showing how they were derived from the Delivery 1 workload model
- Cost estimates must use the **AWS Pricing Calculator** and cite the pricing page URL and access date
- The architecture must address every service defined in Delivery 2 — no service may be left without a cloud mapping

Important Guidelines and Considerations

- **Quantitative Justification is Mandatory:** “We chose t3.medium instances” is not acceptable without showing the calculation. Show how many requests/second a single instance can handle, how many instances are needed at peak load, and how auto-scaling bridges the gap between minimum and maximum.
- **Multi-AZ is the Default:** All critical components (databases, load balancers, ECS services) should be deployed across at least two Availability Zones unless there is a specific cost justification for single-AZ. Single-AZ is a conscious trade-off, not a default.
- **Cost Awareness:** Cloud costs can grow unexpectedly. Understand what drives cost in each service (e.g., data transfer costs for CDN, read/write capacity units for DynamoDB, NAT Gateway data processing fees) and reflect this in your estimates.
- **Traceability from Delivery 2:** Every AWS resource in your diagram must trace to a service or component in your Delivery 2 software architecture. If a new component appears that was not in Delivery 2, explain why it is needed at the infrastructure level.
- **20× is a Stress Test:** The growth scenario is not just about adding more instances. Some components scale trivially (stateless services); others hit fundamental limits (single-primary databases, connection pool exhaustion, NAT Gateway throughput). Identify where the real architectural challenges lie at 20×.

Good luck, and enjoy the challenge of making your architecture real — translating an abstract software design into a concrete, cost-justified cloud infrastructure plan is where cloud engineering truly begins!